CODE REVIEW REPORT

TPRM Firmware — ATxmega128A1 EMS

Automated high-effort review · all .c files · V2.9.7

PROJECT	REVIEWED BY	DATE	FIRMWARE VERSION
GLCD_ATxmega128A1_EMS	Claude Code (automated)	2026-06-27	V2.9.7 (branch: main)

PREPARED FOR

bipin.patel@niyama.in

45	33	14	5	4
.C FILES REVIEWED	CANDIDATES FOUND	VERIFIER AGENTS	CONFIRMED BUGS	PLAUSIBLE ISSUES
12				
REFUTED				

Three root-cause defects of **high severity** were confirmed. The most critical allows out-of-range alarm thresholds to be permanently written to EEPROM during the alarm-disable window — corrupting alarm limits across power cycles. A second confirmed bug silently discards `ALM_CONTROL_FLAG_ID` commands when serial comms are disabled, blocking maintenance operations with no error response. A third allows the 300-second bypass timer to be extended indefinitely with no rate limit. Four lower-severity issues and one version-formatting fragility were also confirmed. Twelve candidates were independently refuted.

CONFIRMED BUGS

CRITICAL `UserInterface.c:2809` · `UserInterface.c:2847`

Unclamped out-of-range value written to EEPROM during alarm-disable window

Both `ParameterOperationsOnLowSettings` and `ParameterOperationsOnHighSettings` wrap all range-clamping logic inside `if(!gu8_AlarmLimitCheckDisable)`. However, the `OSWriteEEPromWord()` call that persists the value to EEPROM sits unconditionally after that guard. When the 300-second bypass window is active, a remote host can send a `WRITE_PARAMETER` command with an arbitrary value — 0, a huge positive, or one that inverts the low/high relationship — and it is accepted and permanently stored. After the bypass window expires, the corrupted alarm limit remains in EEPROM across all subsequent power cycles, silently suppressing real alarms or causing continuous false alarms indefinitely.

FIX Move the `OSWriteEEPromWord()` call inside the clamping guard, *or* add a separate range-validity check that runs unconditionally before the write, regardless of the disable flag.

HIGH `SerialInterface.c:368`

ALM_CONTROL_FLAG_ID silently discarded when serial comms are disabled

HIGH `PeriodicProcess.c:50` · `SerialInterface.c:435`

Alarm-disable timer extendable indefinitely — no rate limit or maximum lifetime

MEDIUM `SerialInterface.c:429–433`

Alarm-disable state not persisted to EEPROM — lost on every power cycle or reset

MEDIUM `UserInterface.c:2112–2114`

Version formatting arithmetic breaks silently when patch number reaches 10

PLAUSIBLE ISSUES

PLAUSIBLE `SerialInterface.c:408, 420, 431`

ASCII '0'/'1' parse pattern triplicated — divergence risk on future changes

PLAUSIBLE `UserInterface.c:2809 & 2847`

Alarm-disable guard duplicated in Low and High settings functions — asymmetric bypass risk

PLAUSIBLE `PeriodicProcess.c:50`

Redundant state: gu8_AlarmLimitCheckDisable is fully derivable from the timer

PLAUSIBLE `SerialInterface.c:435` · `PeriodicProcess.c:61`

Two incompatible timer idioms coexist — uint8 vs uint16 overflow trap for future maintainers

REFUTED — NOT BUGS

LOCATION	CANDIDATE CLAIM	REASON REFUTED
`SerialInterface.c:445`	SendPacket called with uninitialized `i=0`	`i` is explicitly initialized to 0 at declaration (line 360). Zero-length ACK is the established pattern for all three control handlers.
`SerialInterface.c:445`	Uninitialized `txDataBuffer` passed to SendPacket	LCD and COM control handlers do the same; pattern is intentional zero-length acknowledgement, not a data payload.
`UserInterface.c:2809`	Non-atomic access to `gu16_AlarmLimitCheckDisableTimer` (ISR vs. task)	The RTC ISR only sets a tick flag (`ms50=1`). Both serial task and periodic task run sequentially in the foreground super-loop — no ISR preemption on the shared variable.
`SerialInterface.c:433`	Non-atomic access to `gu8_AlarmLimitCheckDisable`	Same bare-metal super-loop rationale — variable is only accessed from foreground tasks, never from interrupt context.
`UserInterface.c:2114`	Mutable local `softver` copy is unnecessary	Style observation only — no observable incorrect behavior.
`UserInterface.c:2114`	Should use `%d.%d.%d` printf directly instead of arithmetic	Equivalent to the confirmed version-arithmetic finding above; flagged as

LOCATION	CANDIDATE CLAIM	REASON REFUTED
		a style alternative, not a separate defect.
`PeriodicProcess.c:50`	Timer management should use the existing `u8_resetTimer` pattern	The difference in timer type (`uint16_t` required for 300 ticks) is correct and intentional — refuted as a style note.
`SerialInterface.c:429`	if/else to set timer is unnecessarily verbose	Functionally equivalent to a ternary; no correctness issue.
`SerialInterface.c:435`	Redundant re-read of global just written	Compiler optimizes this away; no observable behavior difference.
`UserInterface.c:835`	Declaration style inconsistency for new globals	Cosmetic only — no runtime effect.
`UserInterface.c:837`	New globals placed in wrong module	Architecture observation — no behavioral consequence in current code.
`UserInterface.c:2114`	Zero-padding of minor version dropped (formatting change)	Acknowledged as a format change but refuted as a standalone defect — subsumed by the confirmed version-arithmetic finding (F5).